

Random Mutant Selection and Equivalent Mutants Revisited

Rowland Pitts
Software Engineering
George Mason University
Fairfax, Virginia
rpitts@gmu.edu

Abstract—Mutation Testing offers a powerful approach to assessing unit test set quality; however, software developers are often reluctant to embrace the technique because of the tremendous number of mutants it generates, including redundant and equivalent mutants. In spite of their large numbers, redundant mutants are relatively innocuous. Recent research indicates that redundant mutants affect a test engineer's work effort only slightly, whereas equivalent mutants have a direct linear impact. Moreover, the time invested analyzing equivalent mutants produces no unit tests. Dominator mutants address the redundancy problem by allowing the tester to write a minimal number of tests. This paper demonstrates empirically that when using randomly selected mutants as test requirements, the probability of drawing a dominator or quasi-dominator is initially double that of a non-dominator, and progressively increases. It also demonstrates that even non-dominator mutants are highly likely to elicit dominator killing tests. Finally it demonstrates that the probability of selecting an equivalent mutant quickly overwhelms all other selections. These observations provide new insight into the effectiveness of random mutant selection, and into the magnitude of the problem posed by equivalent mutants.

Index Terms—dominator mutants, quasi-dominators, random selection, mutation testing, equivalent mutants

I. INTRODUCTION

Mutation testing [1] is a software testing technique in which program variants are created by introducing individual syntactical faults into the original source code. These faulty versions of the program are called *mutants*. When a unit test responds differently to a mutant than it does to the original version, the test has detected and “killed” the mutant version of the program. The greater the number of mutants that the tests can kill, the higher the quality of the test set.

A. Background

Mutation testing generates excessively large numbers of mutants, *equivalent* and *redundant* mutants in particular. Equivalent mutants behave no differently than the original source code and therefore are unkillable. Redundant mutants are killed in large numbers by relatively few tests. The large numbers of mutants produced make mutation testing “computationally prohibitive which hinders its adoption as an industry standard” [2].

Some research suggests that mutation score is an inaccurate measure for two reasons that relate to equivalent and redundant mutants [3], [4]. Because the tester does not know which

mutants are equivalent, or how many remain, it is not possible to know how much work is left. And because of the large numbers of redundant mutants the mutation score can quickly inflate, giving the impression that a substantial amount of work has been done. Therefore, mutation score is neither a good measure of how much work has been done, nor of how much work remains undone. Reducing the number of nuisance mutants is a logical consideration, however some reduction strategies are considered harmful [5], and the limits of reduction strategies have been explored, along with the limits of improvement over random sampling [6].

Recent research indicates that redundant mutants affect a test engineer's work effort only slightly, whereas equivalent mutants have a direct linear impact [7]. Additionally, the time invested analyzing equivalent mutants produces no unit tests. Dominator mutants [8] seek to address the redundancy problem, but they require the identification of all subsumption relationships, which implicitly identifies all equivalent mutants.

One of the benefits of dominator, or minimal, mutants is smaller mutant sets, and consequently smaller test sets [3]. As it turns out, however, the mutation adequate test sets derived by random selection from all mutants are not much larger than those that target only dominator mutants [9]. Recent research has demonstrated that Dominator Mutants together with quasi-dominators comprise a majority of all mutants [10].

B. Subsumption and Dominator Mutants

If a given mutant is nonequivalent (i.e. killable), and each test that kills that mutant also kills a second mutant, then the given mutant *subsumes* the second mutant [11], and the subsumed mutant can be ignored (because killing the subsuming mutant will also kill the subsumed mutant).

A *Mutant Subsumption Graph* is a directed graph in which each node contains one mutant or multiple mutants that subsume each other. Subsumption is otherwise indicated by the out-bound edges, and nodes with no inbound edges are dominator nodes and contain dominator mutants [11]. These graphs are acyclic. Figure 1 depicts a Mutant Subsumption Graph, based on the unit test results depicted in Table I.

Node multiplicity is defined as follows:

- A node that contains only one mutant is *distinguished*. The contained mutant is also distinguished (e.g. mutant

3 in Figure 1 is distinguished; node {3} is also distinguished).

- A node that contains multiple mutants is *indistinguished*. The contained mutants are also indistinguished (e.g. mutants 1 and 2 in Figure 1 are indistinguished; node {1,2} is also indistinguished).

Node multiplicity is significant in an environment in which mutants are randomly selected one at a time, because the higher the number of mutants a node contains, the more likely it is that the node will be targeted.

*Dominator Mutants*¹ are mutants that are either not subsumed or subsumed only by mutants that are killed by, and only by, the same set of tests [8]. In Figure 1, mutants 1, 2 and 3 are dominator mutants. Node {1,2} is an indistinguished dominator node, and node {3} is a distinguished dominator node.

One of the benefits of dominator mutants is smaller mutant sets, and consequently smaller test sets [3]. For example, the sample data in Table I and Figure 1 indicate that targeting mutants 1 or 2, and 3 is sufficient to kill all mutants, and that Tests 1 and 2 comprise a minimal, mutation adequate test set (and Test 3 is superfluous). As it turns out, however, the mutation adequate test sets derived by random selection from all mutants are only slightly larger than those that target only dominator mutants [9].

C. Quasi-Dominator Mutants

A *Quasi-Dominator Mutant* is a non-dominator mutant that is killed only by dominator-killing tests, as in Figure 1 (gray node), which is based on the tests in Table I. Likewise, a node that contains quasi-dominator mutants is a quasi-dominator node. These mutants perform as dominators – to include the simultaneous killing of a dominator node and all its subsumed mutants – and their presence facilitates the rapid depletion of mutants by random selection, such that it is nearly as efficient as selecting specifically dominator mutants [10].

D. Contributions

This paper’s contributions are as follows:

- Demonstrate empirically that when using randomly selected mutants as test requirements, the probability of drawing a dominator or quasi-dominator in average is initially double that of a non-dominator, and progressively increases.
- Demonstrate that even non-dominator mutants have a high probability of eliciting dominator killing tests, and behaving like a dominator
- Demonstrate that the probability of selecting an equivalent mutant quickly overwhelms all other selections

These observations provide new insight into the effectiveness of random mutant selection, and into the magnitude of the problem posed by equivalent mutants.

The rest of this paper is organized as follows. Section II discusses related work. Section III describes the Methodology

¹also called Minimal Mutants [11]

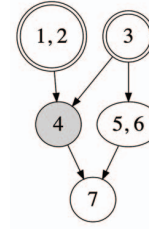


Fig. 1. Mutant Subsumption Graph, defined by Table I. Nodes {1,2} and {3} are dominator nodes (double-circled), and 4 is a quasi-dominator (shaded). It is killed only by tests that kill dominator mutants.

TABLE I
EXAMPLE TESTS AND RESULTS, WHICH CORRELATE TO THE MUTANT SUBSUMPTION GRAPH IN FIG. 1

	Test 1	Test 2	Test 3
Mutant 1:	Kill		
2:	Kill		
3:		Kill	
4:	Kill	Kill	
5:		Kill	Kill
6:		Kill	Kill
7:	Kill	Kill	Kill

followed to conduct the related simulations. Section IV lists results. Section V is discussion, section VI is conclusions, section VII is threats to validity, and section VIII is future work.

II. RELATED WORK

Chusho recognized that exhaustive branch coverage assessment led to an overestimation of quality, and consequently defined the notion of removing redundant, nonessential branches from coverage measures [12]. Analogously, Ammann, Delamaro and Offutt observed that “dynamically subsumed mutants play exactly the same role as non-essential branches do in the Chusho analysis,” and defined an approach to efficiently minimize mutant sets with respect to a set of test cases [3]. Kurtz, Ammann, Delamaro, Offutt and Deng quantified these relationships in Mutant Subsumption Graphs and defined true, static and dynamic subsumption [11]. Kurtz, Ammann and Offutt further showed how symbolic execution can be used to construct static subsumption graphs [8], and characterized subsuming mutants that are not themselves subsumed as *dominators*. Kurtz employed Machine Learning to identify dominator, equivalent, and trivial mutants [13].

Harman and Jia pioneered the idea that a single High Order Mutant (HOM) could stand in for several mutants [14]. Langdon, Harman, Mark and Jia applied subsuming HOMs to relational operators [15]. Lau and Yu demonstrated subsumption relationships in a fault hierarchy, and offered proofs in its support [16]. Kaminski and Ammann defined special HOMs that completely subsumed that hierarchy [17]. Lindström and Márki demonstrated that the fault hierarchies only hold under weak mutation [18].

Zhang, Hou, Hu, Xie, and Mei showed that mutation reduction by operator-based selection does not perform better than

random selection [19]. Zhang, Gligoric, Marinov, and Khurshid demonstrated that operator-based selection and random selection can be effectively applied together [20].

Just, Jalali, Inozemtseva, Ernst, Holmes, and Fraser, demonstrated that mutants are a valid substitute for real faults in software testing [21]. Just, Kapfhammer and Schweiggert made the point that raw mutation scores misrepresented a software system's quality [22], but Sousa demonstrated that mutation score is a fair metric [23]. This effort largely ignores mutation score, and instead focuses on how efficiently a maximum score is reached.

Gopinath, Ahmed, Alipour, Jensen and Groce found that mutant reduction strategies were largely ineffective relative to random sampling, and explored the limits of improvements over random sampling in test subjects that were deemed mutation adequate [5], [6]. This research effort assumes that dominator performance is a textitde facto maximum, and considers the relative performance of random mutant selection, with and without the impediment of equivalent mutants, in test subjects that are known to be mutation adequate.

Chen, Gopinath, Tadakamalla, Ernst, Holmes, Fraser, and Ammann analyzed the relationship between test set size, test adequacy criteria and fault detection [24].

Pitts demonstrated that random selection is nearly as efficient as targeting dominator mutants if equivalent mutants can be avoided, and that it was only fully apparent when working mutation adequate test sets [9]. Pitts also identified quasi-dominators, which are non-dominator mutants that effectively perform as dominators [10].

III. METHODOLOGY

In order to support the overarching objectives, the following research questions will be investigated:

RQ1: What proportion of total mutants do dominator mutants constitute?

RQ2: What proportion of unit tests kill dominator mutants?

RQ3: What is the probability of randomly selecting a dominator mutant?

RQ4: What is the probability of randomly selecting an equivalent mutant?

RQ5: How do the proportions of all mutant types change as testing progresses?

A. Test Subject Selection

The fifteen defects4j [25] subjects listed in Table II were selected because they were previously identified as having fairly comprehensive developer test sets [13], relative to other defects4j subjects, and each has been further scrutinized² to ensure that all previously unkilld mutants were either killed by a newly added test, or determined to be truly equivalent. Eleven of these subjects were previously considered [9], but the number of subjects considered here has been increased by 36%.

²Analysis performed partly within a previous effort [13] and as part of this and other ongoing efforts [9]

TABLE II
JAVA.LANG CLASSES INCLUDED IN THIS STUDY, AND THEIR
CORRELATING DEFECTS4J SUBJECT NUMBERS.

Subject	Class	Mutants
Lang-1	NumberUtils	1679
Lang-2	LocaleUtils	329
Lang-4	LookupTranslator	69
Lang-11	RandomStringUtils	324
Lang-28	NumericEntityUnescaper	122
Lang-33	ClassUtils	753
Lang-45	WordUtils	526
Lang-49	Fraction	1368
Lang-51	BooleanUtils	750
Lang-52	StringEscapeUtils	301
Lang-53	DateUtils	838
Lang-55	StopWatch	178
Lang-59	StrBuilder	2882
Lang-62	Entities	1193
Total		11312

In short, these are the only source code artifacts with accompanying test sets that are known to be truly mutation adequate³. Mutation adequacy is necessary because some of the observations made herein would not otherwise be apparent. The subjects also satisfied other objective criteria, such as being substantial real-world programs. Lastly, three subjects were removed because they were duplicate classes, leaving fifteen, to avoid the impact of redundant scores.

B. Scoring

Mutation score is the normal metric within mutation testing, but because all test sets considered here are mutation adequate, meaning they achieve a score of 100, it will not be the focus of this research effort. The amount of work needed to achieve a score of 100 will be considered instead, and in particular the percentage of all mutants analyzed, or processed, by the tester will be the primary metric.

C. Simulation

In order to simulate a tester who evaluates one mutant at a time [2], the mutants are randomly selected, one at a time, from the entire set, or from an appropriate subset (e.g. dominators and nonequivalent). If the mutant is not equivalent, then one test from among those known to kill it is randomly selected, to simulate a developer writing a test designed to kill that mutant. All mutants known to be killed by the randomly selected test are then removed from the overall set, to account for redundancy and subsumption. All killed mutants are then discarded and the entire process repeated until there are no remaining mutants in the targeted subset of interest. Note that tests that kill the targeted mutant may also kill mutants outside the targeted group. For example, if dominators are targeted, then all the mutants subsumed by the dominator mutant are also removed from the total mutant set, not just dominators. No effort is made to identify a test that will kill the maximum number of mutants, or to realize a minimal or minimum test set.

³At least the only mutation adequate test sets known to the author.

TABLE III
DOMINATOR AND NON-DOMINATOR MUTANT TYPE COUNTS

Subject	Total Mutants	Dominators			Total	Non-Domintor
		Dist.	Ind.	Quasi		
Lang-59	2882	73	758	1285	2116	766
Lang-1	1679	37	588	629	1254	425
Lang-49	1368	38	255	428	721	647
Lang-62	1193	4	401	411	816	377
Lang-53	838	44	234	339	617	221
Lang-33	753	11	263	230	504	249
Lang-51	750	25	390	240	655	95
Lang-25	551	1	302	2	305	246
Lang-45	526	9	64	345	418	108
Lang-2	329	10	113	150	273	56
Lang-11	324	0	120	204	324	0
Lang-52	301	5	76	125	206	95
Lang-55	178	4	37	75	116	62
Lang-28	122	2	31	13	46	76
Lang-4	69	1	6	7	14	55

The simulations are performed using artifacts generated by the Major Mutation Framework [26], [27], on the selected Defects4J [25] subjects, in the Custom Mutants [13] environment, using both the original developer test set and the added test set. In order to achieve a sufficient variety and number of trials, the simulation is repeated twice as many times as there are mutants in each test subject, or aggregate set of subjects.

D. Analysis

Subsumption relationships, and mutant types and counts were determined by programmatically assessing artifacts generated by the Major Mutation Framework [26].

IV. RESULTS

RQ1: What proportion of total mutants do dominator mutants constitute?

Table III contains the breakdown of all dominator types (distinguished, indistinguished, and quasi), and it is also displayed graphically in Fig. 2. 67% of all mutants are dominators, 72% when the average is weighted by the number of mutants in each subject.

RQ2: What proportion of unit tests kill dominator mutants?

Fig. 3 displays the percent of dominator-killing tests in each test subject. 69% of all tests kill dominators, 74% when the average is weighted by the number of mutants in each subject.

RQ3: What is the probability of randomly selecting a dominator mutant?

Distinguished and indistinguished Dominators comprise 39.99% of all mutants, so there is a 0.4 probability of randomly selecting one initially. When quasi-dominators are considered, the total percentage increases to 67% of all mutants (72% when the average is weighted by the number of mutants in each subject). There is a roughly 0.7 probability of randomly selecting a dominator mutant.

RQ4: What is the probability of randomly selecting an equivalent mutant?

Equivalent mutants comprise 11% of all mutants in the study, so there is initially an 0.11 probability of randomly selecting an equivalent mutant.

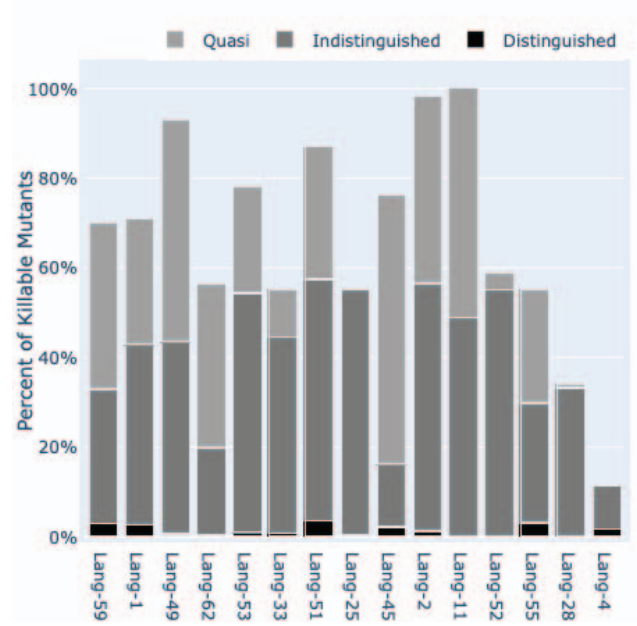


Fig. 2. Distinguished Dominators, Indistinguished Dominators, and Quasi-Dominators, by percentage of all mutants. 67% of all mutants are dominators, 72% when the average is weighted by the number of mutants in each subject.

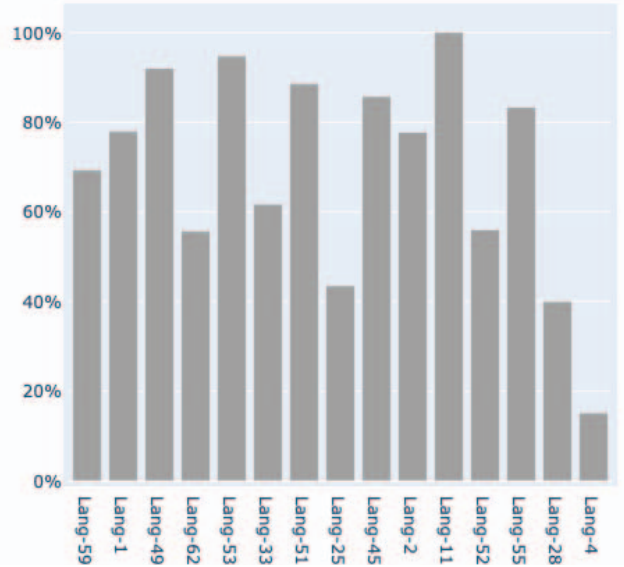


Fig. 3. Dominator Killing Tests by percentage. 69% of all tests kill dominators, 74% when the average is weighted by the number of mutants in each subject.

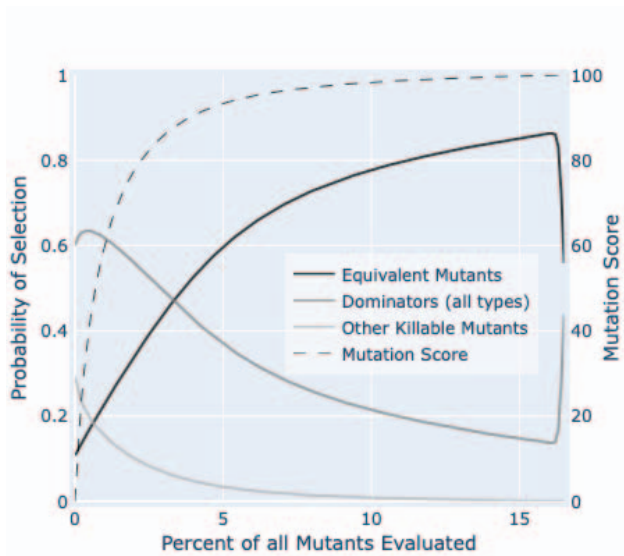


Fig. 4. Probability of Equivalent, Dominator or other Killable Mutant selection, plus Mutation Score, from start to finish (mutation adequacy)

RQ5: How do the proportions of all mutant types change as testing progresses?

Given the nature of redundancy and subsumption, the proportions of mutant types changes throughout the testing cycle. Initially the aggregate of dominator types outnumber everything. As they and other killable mutants are killed and subsume others, the proportion of equivalent mutants increases rapidly. The start-to-finish testing progression is depicted in Fig. 4.

V. DISCUSSION

Dominator mutants together with quasi-dominator mutants comprise a significant portion of total mutants, approximately two-thirds. That means that they are very likely to be randomly selected [10]. In fact, there is no time in the testing cycle at which a dominator of some sort is not most probable (depicted in Fig. 5 without equivalent mutants). Due to the nature of redundancy and subsumption, the probability of selecting a dominator mutant increases as testing progresses.

Fig. 6 aggregates all the dominator types, and compares that probability to other killable mutants. Since it is a binary proposition, the graph is symmetric, but it is also clear how much more likely a dominator is to be selected at the start (roughly double), and that the probability steadily increases for the duration.

When equivalent mutants are considered, the proportions change. Initially equivalent mutants represent a very small proportion (11% on average), but due the nature of redundancy and subsumption, the proportions of mutant types changes throughout the testing cycle, and the proportion of equivalent mutants increases rapidly. The start-to-finish testing progression is depicted in Fig. 4.

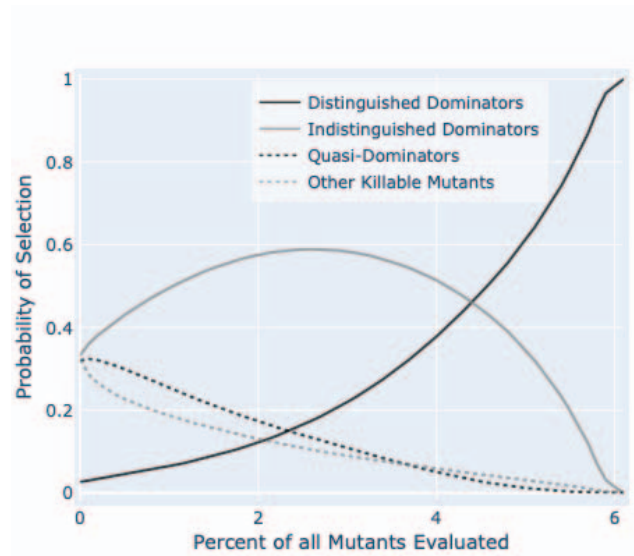


Fig. 5. Probability of Distinguished, Indistinguished, or Quasi-Dominator, or other Killable Mutant selection, from start to finish (mutation adequacy)

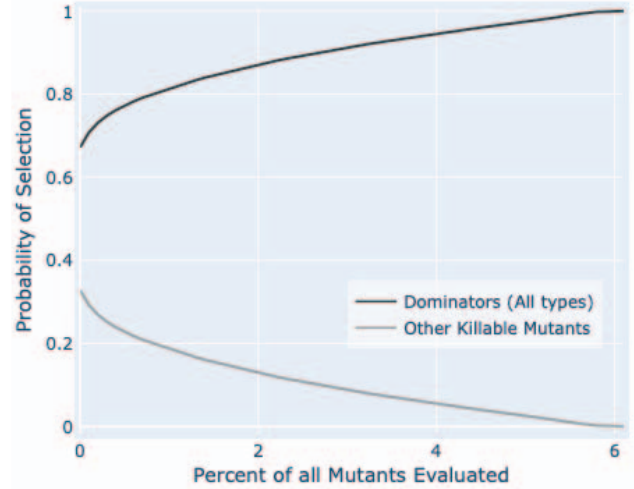


Fig. 6. Probability of Dominator or other Killable Mutant selection (without equivalent mutants), from start to finish (mutation adequacy)

Even non-dominator mutants are likely to elicit a dominator-killing test, effectively performing as dominators. For example, consider the lowest node in Fig. 7 ($\{7,8,9,13,16,17,19,21\}$). Although it is not a dominator node, it can be inferred from the subsumption graph that there are at least 15 dominator-killing tests that also kill it (one for each subsuming dominator). Table IV lists the average percentage of how many non-dominator tests kill the non-dominators in each test subject. The number varies, but in all but one small subject, the majority of tests that kill non-dominators also kill dominators. Therefore, even selecting a non-dominator is very likely to result in dominator-like performance.

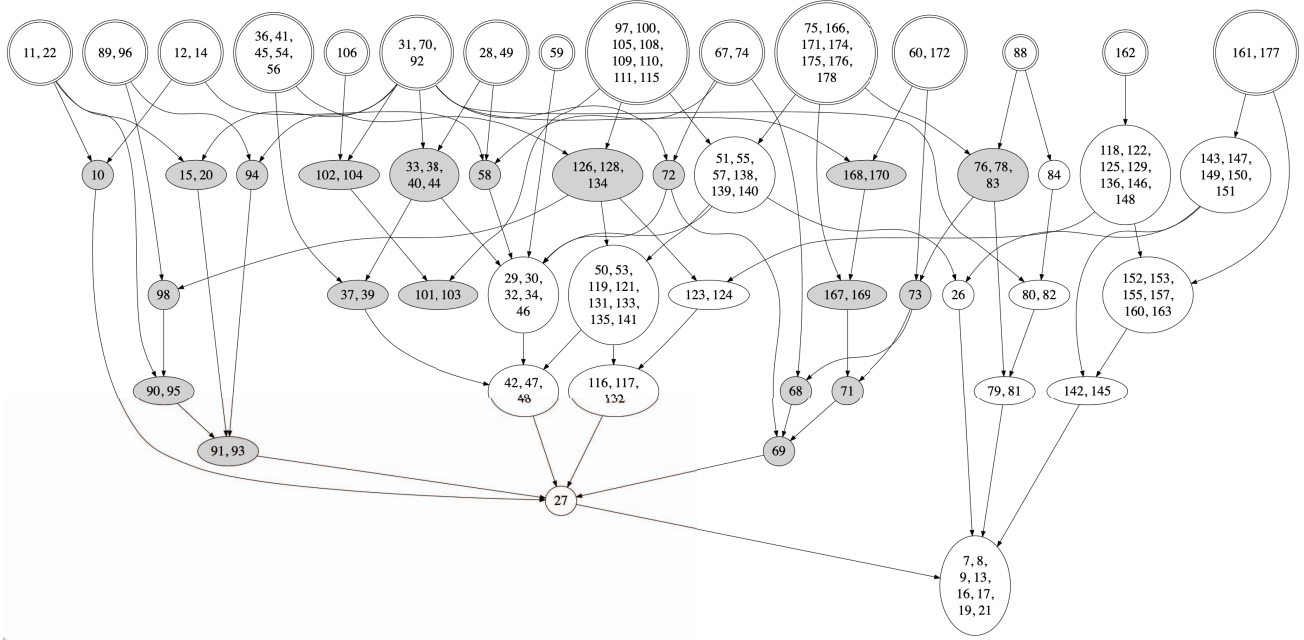


Fig. 7. Subsumption Graph for Lang-55 with Dominator Nodes indicated by double-circles, and quasi-dominator nodes shaded. Quasi-dominator mutants extend the dominator mutants' reach considerably. Lang-55 includes the following 40 equivalent (unkillable) mutants which are not part of the the subsumption hierarchy and therefore are not depicted (1, 2, 3, 4, 5, 6, 18, 23, 24, 25, 35, 43, 52, 61, 62, 63, 64, 65, 66, 77, 85, 86, 87, 99, 107, 112, 113, 114, 120, 127, 130, 137, 144, 154, 156, 158, 159, 164, 165, 173).

A. Random Selection

There are a lot of phenomenon working in favor of random selection:

- The majority of mutants are dominator mutants (Fig. 2 and Table III), making them highly probable for selection.
- The majority of unit tests kill dominators (Fig. 3).
- Due the nature of redundancy and subsumption, regardless of what is selected, the proportion of dominators among the remaining unkilld mutants increases (Fig. 5 and Fig. 6).
- Even non-dominators are predominantly killed by dominator-killing tests, and in that case behave like dominators (Table IV).

B. Equivalent Mutants

Equivalent mutants are a well-known problem. Fig. 4 makes it clear though how quickly equivalent mutants overwhelm the others, and consequently the testing engineer. After evaluating only 5% of all mutants, the mutation score will typically rise above 90, but the proportion of mutants will have shifted such that the tester is almost twice as likely to receive an equivalent mutant, and by ten percent nearly four times as likely. The typical experience is that mutation score rises rapidly but quickly stops rising and the frequency of equivalent mutants accelerates. At that point, testers are likely to stop out of frustration, but with what seems like an accpetable mutation score. The problem is that many non-equivalent mutants remain, particularly dominator (or minimal) mutants.

TABLE IV
PERCENTAGE OF NON-DOMINATOR KILLING TESTS
FOR NON-DOMINATORS

Subject	Total Mutants	Non-Dom. Killing Tests Percent	Std. Dev.
Lang-59	2882	31.8975	0.2494
Lang-1	1679	35.3436	0.7118
Lang-49	1368	29.1313	0.8211
Lang-62	1193	45.9395	2.0847
Lang-53	838	26.151	0.4842
Lang-33	753	35.035	0.1081
Lang-51	750	38.4816	1.188
Lang-25	551	24.3227	1.6405
Lang-45	526	35.1389	0.9802
Lang-2	329	40.6832	3.5036
Lang-11	324	0.0	0.0
Lang-52	301	39.5589	1.0769
Lang-55	178	30.5163	0.3607
Lang-28	122	32.4457	6.1509
Lang-4	69	75.9134	1.1005

Comparing Fig. 9 to Fig. 7 illustrates the difference between a mutation adequate test set and an inadequate test set for Lang-55. There are ten fewer dominator nodes, which gives the impression that only five tests are sufficient, when in fact ten are necessary.

Fig. 8 shows the impact of equivalent mutants on the set of all mutants, and also on dominators. Table V shows the same in more detail. Notice that non-equivalent mutants, when not burdened by equivalent mutants, perform almost as well as dominators alone. Likewise, when dominators are burdened

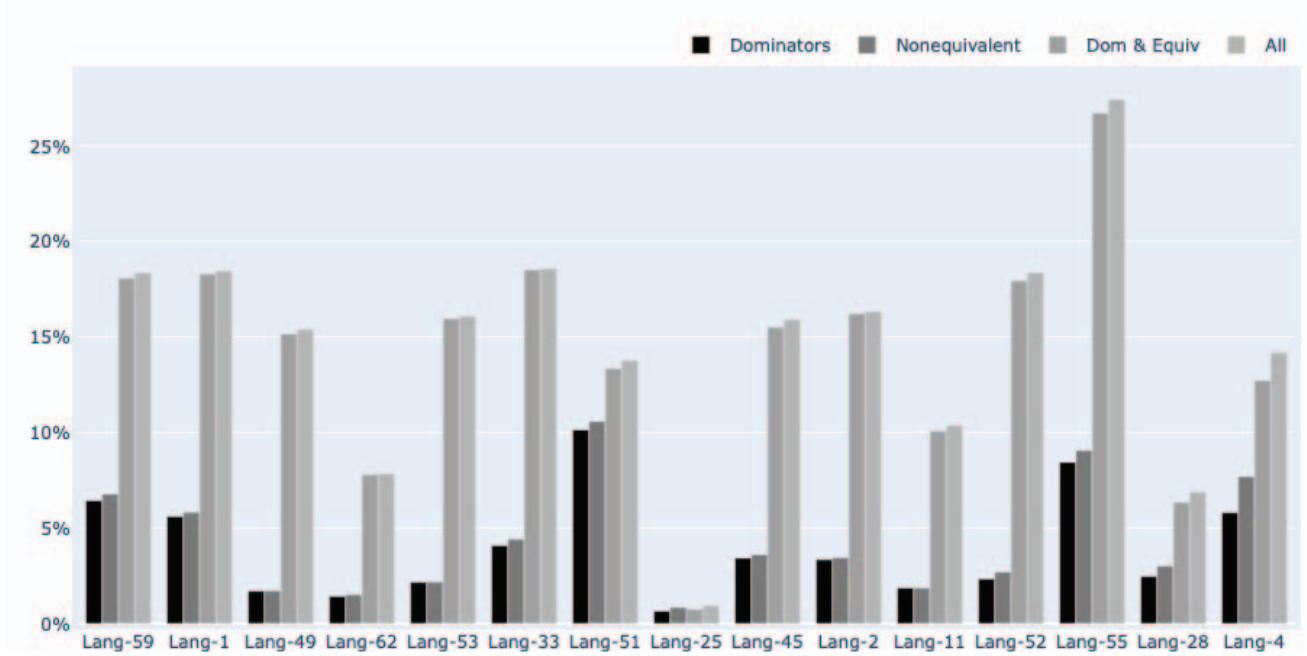


Fig. 8. Percent of mutants processed to reach a mutation score of 100 when randomly selecting from dominators only, nonequivalent (killable) mutants, dominators and equivalent mutants, and from all mutants. Random selection makes a small impact whereas equivalent mutants are very burdensome.

TABLE V
PERCENT OF ALL MUTANTS PROCESSED, WITH STANDARD DEVIATION, TO REACH A MUTATION SCORE OF 100, WHEN RANDOMLY SELECTING FROM DOMINATORS ONLY, NONEQUIVALENT ONLY, DOMINATORS PLUS EQUIVALENT, AND ALL MUTANTS

Subject	Mutants	Trials	Dominators Only		Nonequivalent		Dom And Equiv		All Mutants	
			Percent	Std Dev	Percent	Std Dev	Percent	Std Dev	Percent	Std Dev
Lang-59	2882	5764	6.43	1.3163332E-4	6.77	0.001427508	18.07	0.001030784	18.35	0.002862866
Lang-1	1679	3358	5.60	3.985638E-15	5.82	3.168351E-4	18.29	0.003970694	18.45	0.008474037
Lang-49	1368	2736	1.69	2.5186389E-4	1.71	0.002238677	15.15	0.008067310	15.39	0.002034325
Lang-62	1193	2386	1.42	1.091207E-15	1.51	6.474219E-5	7.79	0.005040539	7.83	0.031651740
Lang-53	838	1676	2.15	1.258696E-15	2.17	6.506480E-4	15.95	0.003994353	16.08	0.006468207
Lang-33	753	1506	4.08	0.0024958249	4.41	6.228197E-4	18.51	0.042433569	18.57	0.014279393
Lang-51	750	1500	10.13	2.523435E-15	10.57	0.002454281	13.35	0.003053503	13.76	0.002777998
Lang-25	551	1102	0.64	0.0027298314	0.84	0.003464404	0.75	0.006144602	0.93	0.015748645
Lang-45	526	1052	3.42	1.191758E-15	3.61	0.005808512	15.50	0.025770393	15.89	0.020324216
Lang-2	329	658	3.34	5.197671E-16	3.44	0.007965579	16.21	0.019877919	16.32	0.031790262
Lang-11	324	648	1.85	5.063095E-16	1.85	5.06310E-16	10.36	0.247810079	10.36	0.165138062
Lang-52	301	602	2.33	3.079512E-16	2.69	0.014744874	17.93	0.014137070	18.35	0.057291153
Lang-55	178	356	8.43	1.037072E-15	9.05	0.003015215	26.72	0.162402844	27.42	0.124796409
Lang-28	122	244	2.46	4.273250E-16	3.01	0.069606489	6.35	0.013360997	6.88	0.020903496
Lang-4	69	138	5.80	7.588220E-16	7.68	0.160607037	12.72	0.343645224	14.17	0.027814626
All	11863	23726	4.31	1.4718603E-5	4.54	2.761612E-4	16.08	6.708601E-5	19.46	6.541417E-4

with equivalent mutants, their performance deteriorates to nearly that of random selection. In both cases, there is only a very small margin separating the two, indicating that the cost of random selection is very small, which is readily apparent in Fig. 8.

It is clear that random selection performs very well when not burdened by equivalent mutants, and dominators perform nearly as poorly in their presence.

It is also important to note that these results depend on mutation adequate test sets for accuracy. For example, Figure 9 depicts the subsumption relations for Lang-55 as defined only by the original developer tests, which are not mutation

adequate, and it is vastly different than those depicted for the same subject in Figure 7 above. The original developer test set incorrectly deems killable mutants as equivalent, resulting in a lower mutation score. It implies fewer unit tests are required to kill all the mutants when in fact three times as many are necessary. Lang-55 is small enough to examine statically, but the same phenomenon exist in all test subjects.

VI. CONCLUSIONS

Fig. 6 demonstrates that when using randomly selected mutants as test requirements, the probability of drawing a dominator or quasi-dominator on average is initially double that of

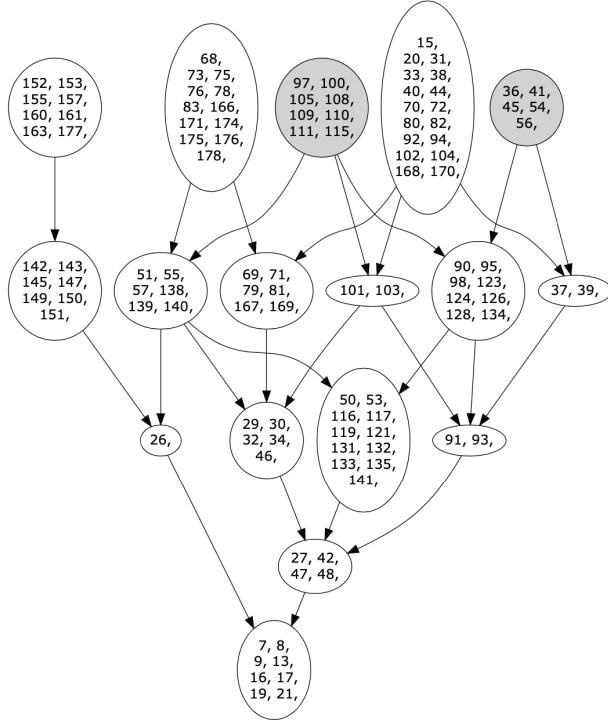


Fig. 9. Subsumption Graph for Lang-55, with incomplete subsumption relationships as defined by the original developer tests. Only two dominator nodes remain intact (shaded).

a non-dominator, and progressively increases. Table IV shows that even non-dominator mutants have a high probability of eliciting dominator killing tests, and consequently behaving like a dominator mutant. Fig. 4 illustrates that the probability of selecting an equivalent mutant quickly overwhelms all other mutant types as testing progresses.

In short, dominators are always more likely to be randomly selected than non-dominators, but equivalent mutants quickly become the most likely to be selected as testing progresses.

These observations provide new insight into the effectiveness of random mutant selection, and into the magnitude of the problem posed by equivalent mutants.

VII. THREATS TO VALIDITY

This study is limited to mutants generated by the Major Mutation framework [26], and its operators. Moreover, because test subjects that are truly mutation adequate are required, this study is limited to these 15 Defects4J subjects, and although they are real world programs, and certainly not toy programs, there are larger testing environments available. Subsumption relationships are relevant to a specific test set [3]; the test sets used herein are composites of each subject's original developer tests and tests added to ensure mutation adequacy, which is a laborious process, and other testers may write tests which result in differing subsumption relationships. Naturally, there is a chance that this cross-section of code does not accurately

represent the full universe of software. Any of these limitations might affect the reliability of projecting these results more broadly.

VIII. FUTURE WORK

The subjects included in this study, plus three with duplicate classes, comprise the only known repository that has been thoroughly analyzed for mutation adequacy. There are many experiments that could be repeated under those conditions, and much more to explore. Increasing the number of test subjects known to be truly mutation adequate is also a logical extension of this work.

REFERENCES

- [1] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, April 1978.
- [2] G. Petrović and M. Ivanković, "State of mutation testing at google," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 163–171. [Online]. Available: <https://doi.org/10.1145/3183519.3183521>
- [3] P. Ammann, M. E. Delamaro, and J. Offutt, "Establishing theoretical minimal sets of mutants," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*, March 2014, pp. 21–30.
- [4] B. Kurtz, P. Ammann, J. Offutt, and M. Kurtz, "Are we there yet? how redundant and equivalent mutants affect determination of test completeness," in *2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, April 2016, pp. 142–151.
- [5] R. Gopinath, I. Ahmed, M. A. Alipour, C. Jensen, and A. Groce, "Mutation reduction strategies considered harmful," *IEEE Transactions on Reliability*, vol. 66, no. 3, pp. 854–874, 2017.
- [6] R. Gopinath, A. Alipour, I. Ahmed, C. Jensen, and A. Groce, "On the limits of mutation reduction strategies," in *Proceedings of the 38th International Conference on Software Engineering*. ACM, 2016.
- [7] B. Kurtz, P. Ammann, J. Offutt, M. E. Delamaro, M. Kurtz, and N. Gökcü, "Analyzing the validity of selective mutation with dominator mutants," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 571–582. [Online]. Available: <https://doi.org/10.1145/2950290.2950322>
- [8] B. Kurtz, P. Ammann, and J. Offutt, "Static analysis of mutant subsumption," in *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, April 2015, pp. 1–10.
- [9] R. Pitts, "Random selection might just be indomitable," in *2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2021, pp. 1–6.
- [10] —, "Quasi-dominators and random selection in mutation testing," in *2022 International Conference on Code Quality (ICCCQ)*, 2022, p. in press.
- [11] B. Kurtz, P. Ammann, M. E. Delamaro, J. Offutt, and L. Deng, "Mutant subsumption graphs," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops*, March 2014, pp. 176–185.
- [12] T. Chusho, "Test data selection and quality estimation based on the concept of essential branches for path testing," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 5, pp. 509–517, May 1987.
- [13] R. Kurtz, "Improving mutation testing with dominator mutants," PhD dissertation, George Mason University, 2018.
- [14] Y. Jia and M. Harman, "Constructing subtle faults using higher order mutation testing," in *2008 Eighth IEEE International Working Conference on Source Code Analysis and Manipulation*, Sep. 2008, pp. 249–258.
- [15] W. B. Langdon, M. Harman, and Y. Jia, "Multi objective higher order mutation testing with genetic programming," *Journal of Systems and Software*, vol. 83, pp. 2416–2430, 12 2010.

- [16] M. F. Lau and Y. T. Yu, "An extended fault class hierarchy for specification-based testing," *ACM Trans. Softw. Eng. Methodol.*, vol. 14, no. 3, pp. 247–276, July 2005. [Online]. Available: <http://doi.acm.org/10.1145/1072997.1072998>
- [17] G. K. Kaminski and P. Ammann, "Using a fault hierarchy to improve the efficiency of dnf logic mutation testing," *2009 International Conference on Software Testing Verification and Validation*, pp. 386–395, 2009.
- [18] B. Lindström and A. Márki, "On strong mutation and the theory of subsuming logic-based mutants," *Software Testing, Verification and Reliability*, vol. 29, no. 1-2, p. e1667, 2019, e1667 stvr.1667. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.1667>
- [19] L. Zhang, S. Hou, J. Hu, T. Xie, and H. Mei, "Is operator-based mutant selection superior to random mutant selection?" in *2010 ACM/IEEE 32nd International Conference on Software Engineering*, vol. 1, 2010, pp. 435–444.
- [20] L. Zhang, M. Gligoric, D. Marinov, and S. Khurshid, "Operator-based and random mutant selection: Better together," in *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE'13. IEEE Press, 2013, p. 92–102. [Online]. Available: <https://doi.org/10.1109/ASE.2013.6693070>
- [21] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. FSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, p. 654–665. [Online]. Available: <https://doi.org/10.1145/2635868.2635929>
- [22] R. Just, G. Kapfhammer, and F. Schweiggert, "Do redundant mutants affect the effectiveness and efficiency of mutation analysis?" *Proceedings - IEEE 5th International Conference on Software Testing, Verification and Validation, ICST 2012*, 04 2012.
- [23] B. Souza, "Is mutation score a fair metric?" in *Proceedings Companion of the 2019 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, ser. SPLASH Companion 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 41–43. [Online]. Available: <https://doi.org/10.1145/3359061.3361084>
- [24] Y. T. Chen, R. Gopinath, A. Tadakamalla, M. D. Ernst, R. Holmes, G. Fraser, P. Ammann, and R. Just, "Revisiting the relationship between fault detection, test adequacy criteria, and test set size," in *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 237–249.
- [25] R. Just. Defects4J website. <http://defects4j.org>. Accessed: 2019-01-21.
- [26] ——. Major mutation website. <http://mutation-testing.org>. Accessed: 2019-01-21.
- [27] ——. "The major mutation framework: Efficient and scalable mutation analysis for java," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA 2014. New York, NY, USA: ACM, 2014, pp. 433–436. [Online]. Available: <http://doi.acm.org/10.1145/2610384.2628053>